



AI CONTEXT PROJECT

Phase 1 – Week 4 Technical Documentation

Field	Details
Project	AEV Platform — Pod Delta (M4)
Squad	AI Context
Squad Lead	Shanmukha Satya Yandra
Team Members	Tanaya Kulkarni; Tanmayee Sunil Thool; Sreevasta Vaddi
Week	Week 4 — Sep 25, 2026 to Oct 1, 2026
Status	Completed through Week 4
Document Date	29 September 2026

Prepared for Week 4 Completion Submission



Table of Contents

1. Introduction
2. Objective
3. Week 4 Scope and Task Allocation
4. Technology Stack and Existing Architecture
5. Memory API Polish
6. User Memory Isolation Re-Verification
7. Real-Time RAG Index Freshness
8. Freshness Monitoring
9. Testing and Verification
10. Issues Encountered and Resolutions
11. Week 4 Deliverables and Completion Status
12. Conclusion



1. Introduction

The AI Context project is a core component of the AEV Platform and is responsible for providing relevant context to Copilot features. During the earlier weeks, the AI Context Squad established the backend service, developed the initial RAG pipeline, implemented asset retrieval and summary capabilities, and introduced a memory and prompt-template layer.

Week 4 focused on hardening and improving the context layer. The main areas were memory API polishing, re-verification of user-memory isolation, real-time RAG index freshness, freshness monitoring, testing, and documentation.

2. Objective

The primary objective of Week 4 was to improve the reliability and freshness of the AI Context service.

- Provide API-level GET and DELETE operations for user memory.
- Re-verify that one user's memory cannot be accessed by another user.
- Process AssetUpdated, ExposureDetected, and RiskCalculated events.
- Trigger re-embedding/index updates for relevant changes.
- Maintain the planned RAG freshness target of within 60 seconds.
- Measure actual index lag using a freshness monitoring/test harness.
- Complete testing and technical documentation for the Week 4 work.

3. Week 4 Scope and Task Allocation

Task	Description	Est. Time	Owner
/copilot/memory GET/DELETE	View and clear memory through the API; no UI.	1 day	Sreevasta Vaddi
Isolation re-verification	Confirm one user cannot access another user's memory.	0.5 day	Sreevasta Vaddi
Real-time freshness	Handle AssetUpdated / ExposureDetected / RiskCalculated events and re-embed within 60 seconds.	1.5 days	Shanmukha Satya Yandra
Freshness monitoring	Test harness to measure actual RAG index lag.	1 day	Tanaya Kulkarni
Tests + documentation	Validation, integration checks and documentation.	1 day	Tanmayee Sunil Thool



4. Technology Stack and Existing Architecture

Week 4 builds on the RAG and memory components developed in the earlier phases. The reference Week 2 technical document records the following technology stack.

Technology	Purpose
Python	Backend programming language
FastAPI	REST API framework
Uvicorn	ASGI server
PostgreSQL	Relational database
pgAdmin 4	Database administration
Visual Studio Code	Development environment
pgvector	Storage and similarity search for vector embeddings

The earlier RAG flow was: Asset Data → Embedding Generation → Vector Index → Similarity Search → Asset Search Endpoint → Asset Summary Prompt → AI Response. Week 4 extends this architecture by introducing a freshness path so that relevant changes can reach the retrieval layer without waiting for a full refresh.

5. Memory API Polish

The memory component was introduced in Week 3 to store useful user-specific context. In Week 4, the API layer was polished around viewing and clearing memory.

5.1 GET Operation

- Retrieves memory associated with the authenticated/requesting user.
- Returns only memory belonging to the appropriate user context.
- Supports the AI Context service in obtaining previously stored context.

5.2 DELETE Operation

- Allows stored user memory to be cleared through the API.
- The delete operation is scoped to the requesting user's memory.
- No user interface was required as part of the Week 4 scope.

The memory API therefore provides a controlled API-level interface for viewing and clearing stored context while preserving user-level separation.



6. User Memory Isolation Re-Verification

Memory isolation is an important security requirement because the memory layer stores user-specific context. Week 4 included a dedicated re-verification of this behaviour.

Expected isolation behaviour:

- User A can read User A's memory.
- User A cannot read User B's memory.
- User A cannot modify User B's memory.
- User A cannot delete User B's memory.

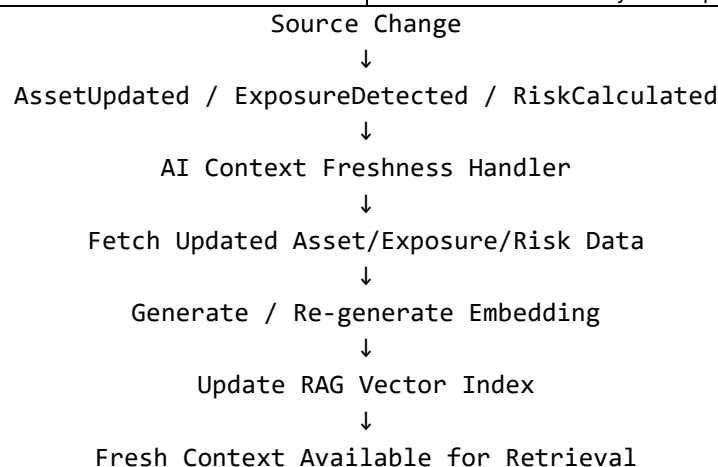
The verification is based on associating memory operations with the appropriate user identifier and checking that database/API queries remain user-scoped.

7. Real-Time RAG Index Freshness

The major Week 4 technical task was the real-time freshness pipeline. The purpose of this pipeline is to reduce the delay between a change in asset/security information and the availability of the updated information in the RAG index.

7.1 Events Covered

Event	Purpose in Freshness Flow
AssetUpdated	Indicates that asset information has changed and may require updated embedding/index data.
ExposureDetected	Indicates a new or changed exposure that should be reflected in retrieved context.
RiskCalculated	Indicates that risk information has changed and the searchable context may need updating.





7.3 Freshness Requirement

The Week 4 execution plan specifies that relevant events should result in re-embedding within 60 seconds. This 60-second target was used as the acceptance criterion for the freshness work.

8. Freshness Monitoring

A monitoring/test harness was included in Week 4 to measure actual RAG index lag. The objective is to measure the elapsed time between a source-data change event and the corresponding searchable/indexed update.

Monitoring sequence:

- Generate or receive a representative asset/exposure/risk change event.
- Record the event timestamp.
- Process the freshness pipeline.
- Record when the updated representation becomes available in the RAG index.
- Calculate the index lag.
- Compare the measured lag with the 60-second target.

This monitoring approach provides a measurable way to validate the freshness requirement rather than relying only on functional API testing.

9. Testing and Verification

Testing for Week 4 covered the following areas:

Area	Verification
Memory GET	Verified retrieval of user-specific memory through the API.
Memory DELETE	Verified clearing/removal of the appropriate memory records.
Memory isolation	Verified that user-specific memory remains isolated between users.
Event handling	Verified handling of AssetUpdated, ExposureDetected and RiskCalculated events.
Re-embedding	Verified that relevant changes trigger embedding/index update processing.
Freshness	Measured index lag using the freshness monitoring/test harness.
Integration	Checked that the updated context remains usable by the Copilot/RAG flow.



10. Issues Encountered and Resolutions

The earlier Week 2 technical documentation recorded several development/configuration issues that were resolved during implementation. These are included here as project context because Week 4 builds on the same service.

Issue	Resolution
Docker configuration/startup problem	Corrected Docker configuration and rebuilt the container.
Pydantic Field import error	Imported Field correctly from Pydantic.
Python indentation/import problems	Corrected the affected Python files.
Vector search verification	Verified pgvector/HNSW configuration and ranked similarity results.

For the Week 4 freshness work, validation was additionally focused on event handling, re-embedding, index update timing and user-memory isolation.

11. Week 4 Deliverables and Completion Status

Deliverable	Status
Memory GET/DELETE API	Completed
User-memory isolation re-verification	Completed
Real-time freshness event flow	Completed
AssetUpdated / ExposureDetected / RiskCalculated handling	Completed
RAG re-embedding/index update flow	Completed
Freshness monitoring/test harness	Completed
Testing and integration validation	Completed
Week 4 technical documentation	Completed

11.1 Overall Progress Through Week 4

The AI Context Squad has completed the project work scheduled through Week 4. The earlier phases established the searchable asset/RAG foundation and memory/prompt capabilities; Week 4 strengthened those capabilities through API polishing, isolation verification and real-time retrieval freshness.



Phase	Major Capability	Status
Week 1	Backend foundation, prompt/template groundwork and initial asset-search capability	Completed
Week 2	RAG pipeline, embeddings, HNSW vector search, asset search and asset-summary prompt	Completed
Week 3	Memory layer, user isolation, memory types/expiry and prompt-template management	Completed
Week 4	Memory API polish, isolation re-verification, real-time RAG freshness and monitoring	Completed

12. Conclusion

Week 4 completed the planned hardening work for the AI Context service. The memory API was refined for viewing and clearing user-specific context, user-memory isolation was re-verified, and a real-time freshness flow was implemented for relevant asset, exposure and risk changes.

The freshness monitoring work provides a measurable method for checking the planned 60-second RAG freshness target. Together with the RAG pipeline, memory layer and prompt management capabilities completed in the earlier weeks, the Week 4 work provides a stronger and more up-to-date context layer for the AEV Platform Copilot.